Here is the repository review based on the provided files (002-repository-review-prompt.md and tables.md).
Note: As specified in the review prompt guidelines ("If you cannot see something, say so; do not assume"), because the codebase files (README.md, crawler/crawler.go, extract/wikipedia.go, extract/text.go, main.go, crawler/crawler_test.go, extract/extract_test.go, decision logs, specs, and constitution) and docs/assignment-and-rubric.md were not attached in this session, code-level bug identification, exact rubric scoring, and README text comparisons cannot be fully verified. The evaluation below addresses what is visible in tables.md and structures the review criteria accordingly.

1. Score Each Rubric Criterion

- **Status: Cannot be scored** (source files and rubric not attached).
- **Evidence:** The prompt specifies five rubric criteria (20 points each), but docs/assignment-and-rubric.md was not attached. To assign scores, a grader would need to verify test coverage in crawler_test.go and extract_test.go, concurrency safety in crawler.go, DOM cleaning logic in wikipedia.go / text.go, and clean exit handling in main.go.

2. Assignment Steps S1–S10 (docs/assignment-and-rubric.md)

- **Status: Cannot be verified** (rubric and specification documents not attached).
- **Evidence:** Steps S1 through S10 from docs/assignment-and-rubric.md are not visible in the current workspace. Verification requires inspecting specs/001-wikipedia-scraper/spec.md, docs/decisions.md, and execution artifacts.

3. Risky Code (crawler.go, wikipedia.go, text.go, main.go)

- **Status: Cannot be evaluated** (source code files not attached).
- **Evidence:** Without access to crawler.go, wikipedia.go, text.go, and main.go, specific concurrency race conditions, DOM parsing edge cases, and non-zero exit code behaviors cannot be identified with concrete inputs.

4. README Conclusions vs. results/tables.md

- **Status: Inconclusive** (README.md not attached).
- **Evidence:** Because README.md was not attached, we cannot quote its text or compare its written conclusions against results/tables.md. However, analyzing results/tables.md alone yields the following empirical baselines:
  - **Live Crawl Performance (Table 2):** Crawling and writing 10 pages takes **0.56 s** sequentially (1 request at once) versus **0.33 s** concurrently (4 requests at once), yielding a **1.7× speed-up** with peak memory increasing marginally from **41 MB to 48 MB**.
  - **Offline Benchmarks (Table 3):** With 100 ms simulated network latency per page, throughput scales from 9.3 pages/s (parallel=1) up to **43.9 pages/s** (parallel=8), demonstrating near-linear concurrency scaling. Extraction alone (WikipediaArticle-10)

runs at **255.8 pages/s** (39 ms per crawl).
    - **Extraction Yield (Table 4):** Clean extraction retains between **29% and 47%** of Scrapy-style raw character counts across the 10 target Wikipedia pages (totaling 276,784 clean characters vs 751,602 raw characters).

5. Grader Friction

- **Status: Identified structural risks** based on submission requirements.
- **Where:** Repository setup and execution entry point (main.go).
- **Problem:** Graders cloning repositories frequently trip over missing configuration variables, uninitialized output directories (results/), or missing contact information (such as a required polite User-Agent contact e-mail for Wikipedia scraping via Colly).
- **Suggested Change:** Ensure main.go automatically creates output directories (e.g., results/) if absent, and verify that Colly sets a descriptive User-Agent header containing a valid contact email to prevent HTTP 403 blocks from Wikimedia servers.

6. Top 3 Actionable Changes (<20 Minutes)

1. **Verify Output Directory Creation (main.go / crawler.go)**
    - *Severity:* High
    - *Where:* File writing initialization
    - *Problem:* If the results/ directory does not exist locally when a grader runs go run main.go, the program will crash on startup when attempting to open results/wikipedia.jl.
    - *Suggested Change:* Add os.MkdirAll("results", 0755) prior to opening file handles.
2. **Explicit Exit Codes (main.go)**
    - *Severity:* Medium
    - *Where:* main() return path
    - *Problem:* Silent error swallowing or returning os.Exit(0) on pipeline failures frustrates automated grading scripts.
    - *Suggested Change:* Ensure non-zero exit codes (os.Exit(1)) are explicitly returned if crawler or parser errors occur.
3. **Check JSON Lines Formatting (results/wikipedia.jl)**
    - *Severity:* Medium
    - *Where:* Output serialization
    - *Problem:* Malformed JSON lines (trailing commas, missing newlines) break automated ingestion scripts.
    - *Suggested Change:* Validate that every record written via json.Marshal is followed immediately by a newline character \n.

Would you like to review specific code snippets or test files from your local workspace to complete the rubrics for S1–S10?